

## Introduction

---

Every interviewer eventually asks some version of "give me the next best arrangement of this."

Sometimes it's phrased as scheduling, sometimes as a combination lock, sometimes — like here — as a string problem. LeetCode 3720, *Lexicographically Smallest Permutation Greater Than Target*, is a great vehicle for testing whether a candidate actually understands the **next permutation** pattern or has just memorized `itertools.permutations`.

The reason this problem is worth your time isn't the specific string manipulation. It's that the underlying technique — find the last position you can still improve, make the smallest possible improvement there, then minimize everything after it — shows up constantly: task scheduling, key ordering, combinatorial search, even the C++ standard library's `next_permutation`. Once you internalize this pattern, an entire class of "find the next X" problems stops being scary.

## Problem Overview

---

You're given two lowercase strings of equal length: `s`, which you're allowed to freely rearrange, and `target`, which is fixed and never changes. Your job is to rearrange the letters of `s` into the **lexicographically smallest** string that is **strictly greater** than `target`. If no rearrangement of `s` can beat `target`, return an empty string.

In plain terms: you have a bag of letters, and someone set a benchmark you need to beat. You want to beat it by the smallest possible margin — not overshoot with something way bigger than necessary.

## Example

---

Let `s = "abc"` and `target = "bba"`.

All permutations of `abc`: `abc`, `acb`, `bac`, `bca`, `cab`, `cba`.

Compare each against `bba`:

### Permutation Bigger than "bba"?

|                  |     |
|------------------|-----|
| <code>abc</code> | No  |
| <code>acb</code> | No  |
| <code>bac</code> | No  |
| <code>bca</code> | Yes |

## Permutation Bigger than "bba"?

|     |     |
|-----|-----|
| cab | Yes |
| cba | Yes |

The smallest one that qualifies is `bca`. That's the answer.

Notice something important: `s` and `target` don't need to share the same *count* of each letter. In this example they happen to use the same letters, but in general `target` can be built from entirely different letters or different quantities — which is exactly what makes this problem harder than a plain "next permutation" question.

## Intuition

---

Start with the simpler version of this problem: given a single number, find the next-largest permutation of its own digits. This is the classic interview warm-up, and it's worth understanding fully before tackling the letter-frequency version.

Picture the digits from right to left. As long as each digit is greater than or equal to the one after it, that suffix is already at its maximum value — no rearrangement of it can produce something bigger. The moment you find a digit smaller than something to its right, you've found the **pivot**: the one position where an improvement is actually possible.

Once you have the pivot, you don't want to jump to just *any* bigger arrangement — you want the smallest possible one that's still bigger than the original. So you look at everything right of the pivot and pick the smallest value that's still larger than the pivot digit. Swapping it in gives you the smallest possible increase.

After the swap, the tail (everything right of the pivot) is still sorted in decreasing order — the *largest* possible arrangement of those digits. Since you want the *smallest* possible overall result, you reverse that tail, turning it into ascending order.

This exact routine — scan for the pivot, swap in the smallest valid upgrade, reverse the tail — is what `next_permutation` does under the hood in the C++ standard library.

Now, the twist in this problem: `target` isn't guaranteed to be a rearrangement of `s`. It can have completely different letters, in different amounts. That means you can't scan `target`'s own layout for a pivot the way you would with digits — you first have to ask, at every prefix length, "do I even have enough of the right letters in `s` to match `target` this far?" That's the extra bookkeeping this problem adds on top of the classic pattern.

## Brute Force Solution

---

**Idea:** Generate every permutation of `s`, filter to the ones strictly greater than `target`, sort what's left, and take the smallest.

**Algorithm:** 1. Generate all permutations of `s`. 2. Discard duplicates (if `s` has repeated letters) and any permutation not strictly greater than `target`. 3. Sort the survivors and return the first one.

**Advantages:** Trivial to reason about, easy to verify correctness, no edge-case handling needed.

**Disadvantages:** Permutation count grows as  $n!$ . Even a 10-character string produces over 3.6 million permutations. With `s` allowed up to length 300 in this problem, brute force isn't just slow — it's computationally impossible.

**Time complexity:**  $O(n! \cdot n \log n)$  (generating and sorting permutations). **Space complexity:**  $O(n! \cdot n)$  to store them all.

## Optimal Solution

---

Since checking "is there a bigger letter available" only ever involves 26 lowercase letters, we can replace expensive permutation generation with a **letter-count** approach — the frequency-based cousin of the pivot-scan trick.

**Step 1 — Set up the counts.** Count every letter in `s` using a `Counter`. This is the "bag of letters" you're allowed to draw from.

**Step 2 — Pre-consume target's prefix.** Before scanning, subtract out every letter of `target` except the last character, as if you were tentatively building `target` itself out of `s`. Track a `neg` counter: every time a letter's count drops below zero, `neg` increases. `neg > 0` means "this prefix isn't actually buildable from `s`."

**Step 3 — Walk backward from the last position.** At each position `i`, moving from the end toward the start:

- If `neg == 0`, the prefix up to (but not including) position `i` is genuinely buildable from `s`. Now search upward from `target[i]` through the alphabet for the smallest letter you still have in stock that's bigger than `target[i]`.
- If found: use it at this position, then append every remaining letter you're holding in ascending order. Return the result immediately — this is the answer.

- If `neg != 0`, or no bigger letter was available at this position, this prefix doesn't work. Give `target[i]`'s letter back to your count (undoing the pre-consumption), decrement `neg` if that letter had caused a shortfall, and move one position to the left to try a shorter prefix.

**Step 4 — No match found.** If you walk all the way through without returning, no permutation of `s` can beat `target`. Return an empty string.

Here's the mapping back to the digit version, so the pattern clicks:

### Digit version

Scan right-to-left for a digit smaller than something after it

Find smallest tail digit bigger than pivot

Reverse the tail

### Letter-frequency version

Scan right-to-left for a position where the prefix is still buildable ( `neg == 0` )

Search upward from `target[i]` for the smallest letter you have in stock

Append remaining letters in ascending order

## Python Code

```
from collections import Counter

def smallest_string_greater_than_target(s: str, target: str) -> str:
    n = len(s)
    counts = Counter(s)
    neg = 0 # number of letters currently short of what's needed

    # Tentatively "use" target's letters for every position except the last
    for i in range(n - 1):
        counts[target[i]] -= 1
        if counts[target[i]] < 0:
            neg += 1

    # Walk backward, looking for the rightmost position we can improve
    for i in range(n - 1, -1, -1):
        if neg == 0:
            # This prefix is buildable; try to find a bigger letter here
            for c in range(ord(target[i]) + 1, ord('z') + 1):
                letter = chr(c)
                if counts[letter] > 0:
                    counts[letter] -= 1
                    remaining = sorted(counts.elements())
                    return "".join(target[:i]) + letter + "".join(remaining)

            # No improvement possible here; undo and try a shorter prefix
            if i > 0:
                counts[target[i - 1]] += 1
                if counts[target[i - 1]] > 0:
```

```

        neg -= 1

    return ""

```

## Code Walkthrough

- `Counter(s)` builds the letter inventory we're allowed to draw from.
- The first loop pre-consumes `target[0:n-1]` from that inventory, simulating "what if we tried to exactly match target up to the last character." Any letter going negative means that prefix isn't buildable, so `neg` tracks how many such shortfalls currently exist.
- The second loop walks `i` from `n-1` down to `0`. `neg == 0` is the "is this prefix even reachable" check — the frequency equivalent of the pivot condition in the digit version.
- The inner `for c in range(...)` loop searches upward through the alphabet from `target[i]`, which is the frequency-based version of "find the smallest tail value bigger than the pivot."
- `sorted(counts.elements())` produces every remaining letter in ascending order — this replaces the "reverse the tail" step, since sorting ascending is the smallest possible arrangement of what's left.
- If a fix at position `i` doesn't work, we put `target[i-1]`'s letter back into `counts` (undoing our earlier tentative consumption) and adjust `neg`, then let the loop retry with a shorter prefix.
- Falling through both loops without returning means no valid answer exists, so we return `""`.

## Dry Run

Let's trace `s = "abc"`, `target = "bba"`.

**Setup:** `counts = {a:1, b:1, c:1}`.

Pre-consume `target[0:2] = "bb"` :- Subtract `b`: `counts[b] = 0` .- Subtract `b` again:  
`counts[b] = -1 → neg = 1`.

**Main loop,** `i` from 2 down to 0:

- `i = 2`: `neg = 1`, skip the search. Undo: add back `target[1] = 'b' → counts[b] = 0`. Since it's no longer negative, `neg = 0`.
- `i = 1`: `neg = 0`. Search upward from `target[1] = 'b'` (i.e., from `'c'` onward).  
`counts[c] = 1 > 0` — found it. Use `c`: `counts[c] = 0`. Remaining letters: `{a:1, b:0, c:0}` → sorted elements = `['a']`. Result = `target[:1] ("b") + "c" + "a" = "bca"`.

Return `"bca"` — matches the expected answer from the example.

## Complexity Analysis

---

**Time:**  $O(n \cdot 26)$ . For each of the `n` positions, the inner alphabet search does at most 26 checks. Building the sorted remainder is  $O(n \log n)$  once, at the point we return. Overall:  **$O(n \log n)$** , dominated by the final sort — but crucially, this happens *once*, not per iteration.

**Space:**  $O(26)$  for the letter counts, plus  $O(n)$  for the output string. Effectively  **$O(n)$** .

This is correct because the alphabet size is a fixed constant (26), so the per-position work never grows with input size — only the single sort at the end scales with `n`.

## Alternative Solutions

---

You could sort `s` and binary-search for a valid arrangement at each prefix length, but this adds complexity without improving the asymptotic bound — the letter-count approach already does the minimum necessary work per position. Another option is recursively trying every prefix length with backtracking, but that reintroduces exponential blowup in the worst case. The greedy frequency-count method is both the simplest and asymptotically optimal choice here.

## Edge Cases

---

- **`s` and `target` share no letters in common at any usable position:** the algorithm naturally falls through to returning `""`.
- **`s` has repeated letters:** `Counter` handles this correctly without generating duplicate permutations, unlike the brute-force approach.
- **`target` is lexicographically larger than every possible permutation of `s`:** the loop exhausts all positions without finding a valid swap, returning `""`.
- **`s` is a permutation of `target` itself:** this degenerates cleanly into the classic single-string next-permutation problem.
- **Single-character strings:** `n = 1` means the pre-consumption loop does nothing, and the main loop either finds a bigger letter or returns `""` immediately.

## Common Mistakes

---

1. **Generating all permutations** — works for tiny inputs, times out immediately at scale.

2. **Forgetting to sort the remaining letters ascending** — appending them in arbitrary order can produce a larger-than-necessary result.
3. **Not tracking `neg` correctly when undoing a tentative letter usage** — forgetting to decrement `neg` when a letter count returns to non-negative causes false rejections later.
4. **Assuming `s` and `target` share letter frequencies** — they don't have to, and code that assumes this will silently produce wrong answers instead of crashing.
5. **Searching for "any bigger letter" instead of the smallest bigger letter** — this produces a valid answer but not the lexicographically smallest one.
6. **Off-by-one errors in the prefix pre-consumption** — consuming `target[0:n]` instead of `target[0:n-1]` throws off the entire alignment between prefix length and position index.

## Interview Questions

---

- How would this change if `s` and `target` could contain uppercase letters or digits too?
- What's the time complexity if the alphabet size were much larger, say Unicode?
- Can you solve the simpler version — where `s` is a permutation of `target`'s own digits — without the `Counter` bookkeeping?
- How would you find the *k*-th smallest permutation greater than `target` instead of just the first?
- What changes if you needed the largest permutation *smaller* than `target` instead?

## Similar Problems

---

- **LeetCode 31 — Next Permutation:** the pure digit/array version of this exact pivot-swap-reverse pattern, no letter-frequency complication.
- **LeetCode 46 — Permutations:** foundational permutation generation, useful for understanding why brute force explodes.
- **LeetCode 60 — Permutation Sequence:** finding the k-th permutation directly, without generating all previous ones.
- **LeetCode 556 — Next Greater Element III:** the next-permutation pattern applied directly to integers.

## Key Takeaways

---

The core lesson is a reusable pattern, not a one-off trick: find the rightmost position where improvement is still possible, make the *smallest* valid improvement there, then minimize everything after it. Whether you're working with digits in a fixed sequence or letter counts in a bag, the shape of

the solution stays the same. Recognizing this pattern turns an intimidating combinatorics problem into a straightforward linear scan.

## Watch the Video

---

For the full walkthrough — including a live timing comparison between brute force and the optimal solution, and a two-question quiz to check your understanding — watch the video here: {LINK}

## About the Series

---

This breakdown is part of **Fun with Learning Technology**, a series dedicated to building real algorithmic intuition rather than memorized solutions. Each episode takes a coding interview problem, strips away the intimidating parts, and rebuilds it from first principles — so the pattern sticks long after the specific problem is forgotten.

## Call To Action

---

If this explanation helped the pivot-swap-reverse pattern click for you, give the video a like on YouTube and subscribe for more interview-focused breakdowns. For deeper dives and companion write-ups delivered straight to your inbox, subscribe to the Substack. Drop a comment with your own approach or questions, and share this with anyone prepping for coding interviews.

---

## Quick Review

---

### When target permutation isn't achievable, what's the go-to pattern?

Next-permutation-style suffix rearrangement: scan from the right, find the pivot, swap, then sort/reverse the tail.

### Time complexity for this permutation-search approach?

$O(n \log n)$  per query (or  $O(n)$  with counting sort on digits), dominated by sorting the suffix after the pivot.

### Space complexity for the pivot-swap approach?

$O(n)$  for the array/string copy of the permutation being built;  $O(1)$  extra if done in place.



**Trickiest edge case in this problem?**

No pivot exists (array is already the largest arrangement) — must correctly report 'no valid permutation' instead of wrapping around.

**This pivot/suffix technique vs brute-force permutation generation — when to prefer which?**

Pivot technique for large  $n$  (avoids factorial blowup); brute-force only viable for tiny fixed digit counts like the 3-digit example.

**Follow-up: how would you extend this to find the smallest permutation greater with duplicate digits allowed?**

Use a multiset/count array for the suffix instead of distinct swaps, picking the smallest value strictly greater than the pivot.

---

Lexicographically Smallest Permutation Greater Than Target — free Python cheat sheet